جلسهٔ ۱۰ — تشخیص و تطبیقِ ویژگی‌ها (Feature Detection & Matching)

جزوهٔ درس — هفتهٔ دهم

نگاهِ کلی به این جلسه

تا اینجا یاد گرفتیم چطور محتوای یک تصویر را توصیف کنیم: لبه‌ها، ناحیه‌ها و شکل‌ها. اما یک پرسشِ بنیادی باقی مانده است: چطور بفهمیم که یک نقطهٔ مشخص در تصویرِ اول، همان نقطه در تصویرِ دوم است؟ اگر از یک کنتور دو عکس بگیریم — یکی صاف و یکی از زاویه و کوچک‌تر — چطور دو عکس را روی هم منطبق کنیم؟ پاسخ، مفهومِ «ویژگی» (feature) است: نقاطِ متمایز و قابلِ‌شناسایی که می‌توان آن‌ها را در تصویرهای مختلف پیدا و با هم مطابقت داد.

یادداشت

ویژگی‌ها پایهٔ «بینایی هندسی» هستند: دوختنِ پانوراما، ردیابیِ اجسام، واقعیتِ افزوده و — برای ما — منطبق‌کردنِ عکسِ یک سند/کنتور روی الگو تا آن را صاف کنیم. این دقیقاً هستهٔ مرحلهٔ بعدیِ پروژهٔ DocScan (هفتهٔ ۱۱، تصحیحِ پرسپکتیو) است.

یک تشبیهِ ساده

فرض کنید می‌خواهید دو عکس از یک ساختمان را — که از دو زاویهٔ متفاوت گرفته شده‌اند — روی هم منطبق کنید. اگر به دوستی بگویید «این گوشهٔ پنجرهٔ سومِ طبقهٔ دو» را در هر دو عکس پیدا کند، او به‌راحتی این کار را می‌کند، چون آن گوشه یک نشانهٔ متمایز است. اما اگر بگویید «این نقطه از دیوارِ یک‌دست» را پیدا کن، ناتوان می‌ماند، چون همهٔ نقاطِ دیوار شبیهِ هم‌اند. کامپیوتر هم دقیقاً همین‌طور است: به نقاطِ متمایز (ویژگی‌ها) و یک «توضیحِ» فشرده از هر نقطه (توصیف‌گر) نیاز دارد تا بتواند تناظر برقرار کند. کلِ این جلسه دربارهٔ همین دو چیز است: چطور نقاطِ متمایز را پیدا کنیم، و چطور آن‌ها را طوری توصیف کنیم که با وجودِ چرخش و تغییرِ اندازه بازشناختنی بمانند.

۱ ویژگیِ خوب چیست؟

هر نقطه‌ای برای تطبیق مناسب نیست. یک نقطه در وسطِ یک دیوارِ صاف و یک‌دست، هیچ نشانهٔ متمایزی ندارد؛ اگر کمی جابه‌جا شویم، منظره فرقی نمی‌کند، پس نمی‌توانیم دوباره «همان» نقطه را پیدا کنیم. یک ویژگیِ خوب سه خاصیت دارد:

- **متمایز (Distinctive):** با محیطِ اطرافش فرق دارد، پس دوباره قابلِ پیداشدن است.

- **تکرارپذیر (Repeatable):** همان نقطهٔ فیزیکی در تصویرهای مختلف (زاویه یا نورِ متفاوت) باز هم شناسایی می‌شود.
- **محلی (Local):** با یک وصلهٔ کوچک توصیف می‌شود، پس در برابرِ پوشیده‌شدنِ بخشی از تصویر مقاوم است.

۱.۱ چرا گوشه؟

سه حالت را در نظر بگیرید و تصور کنید یک پنجرهٔ کوچک را روی تصویر کمی جابه‌جا می‌کنیم:

- **ناحیهٔ صاف:** جابه‌جایی در هر جهت، منظره را تغییر نمی‌دهد ← هیچ مکان‌یابی‌ای ممکن نیست.
- **لبه:** جابه‌جایی در امتدادِ لبه تغییری ایجاد نمی‌کند ← ابهامِ یک‌بُعدی (نمی‌دانیم کجای لبه هستیم).
- **گوشه:** جابه‌جایی در هر جهت منظره را تغییر می‌دهد ← نقطه در دو بُعد کاملاً مشخص است.

نکتهٔ کلیدی

به همین دلیل «گوشه» کلاسیک‌ترین ویژگیِ خوب است: تنها جایی که موقعیتش را به‌طورِ یکتا در صفحه می‌خ‌میخکوبد. آشکارسازِ **هریس** (Harris) دقیقاً همین را می‌سنجد: میزانِ تغییرِ شدت در همهٔ جهت‌ها.

۲.۱ ماتریسِ ساختار (Structure Matrix)

هریس چطور کار می‌کند؟ درونِ یک پنجرهٔ کوچک، گرادیان‌های تصویر (I_x, I_y) را نگاه می‌کنیم و ماتریسِ $\times$ زیر را می‌سازیم:

ریاضیات

$$M = \sum_{\text{پنجره}} \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

دو مقدارِ ویژهٔ این ماتریس، λ_1 و λ_2، می‌گویند وصله در دو جهتِ اصلی چقدر تغییر می‌کند:

- هر دو کوچک ← ناحیهٔ صاف.
- یکی بزرگ، یکی کوچک ← لبه.
- هر دو بزرگ ← **گوشه**.

برای پرهیز از محاسبهٔ مستقیمِ مقادیرِ ویژه، هریس امتیازِ $R = \lambda_1\lambda_2 - k(\lambda_1+\lambda_2)^2 = \det(M) - k\,\mathrm{tr}(M)^2$ را حساب می‌کند ($k \approx 0.04$).

در شکلِ زیر، آشکارسازِ هریس را روی یک خودروی واقعی از مجموعه‌دادهٔ شما اجرا کرده‌ایم. نقاطِ قرمز، گوشه‌های شناسایی‌شده هستند:





چپ: تصویرِ خاکستری راست: گوشه‌های هریس (قرمز) روی محل‌های متمایز

می‌بینید که گوشه‌ها روی محل‌های متمایز (چراغ‌ها، لبه‌های قطعات، محلِ اتصالِ اجزا) می‌نشینند — همان نقاطی که در عکسِ دیگری از همان خودرو هم قابلِ پیداشدن‌اند.

۳.۱ آشکارسازِ FAST، گام‌به‌گام

هریس دقیق است اما کُند؛ برای کاربردهای بی‌درنگ (ویدیو) به چیزِ سریع‌تری نیاز داریم. **FAST** (Features from Accelerated Segment Test) یک آزمونِ بسیار ساده و سریع دارد:

۱. یک دایره از ۱۶ پیکسل را دورِ پیکسلِ موردِنظر p در نظر بگیر.

۲. اگر دستِ‌کم ۱۲ پیکسلِ پیوسته روی این دایره همگی روشن‌تر از p (به‌اندازهٔ یک آستانه t) یا همگی تیره‌تر باشند، آن‌گاه p یک گوشه است.

۳. برای سرعت، ابتدا فقط ۴ پیکسلِ بالا/پایین/چپ/راست بررسی می‌شوند؛ اگر شرط برقرار نباشد، پیکسل فوراً رد می‌شود.

نکتهٔ کلیدی

همین آزمونِ ساده، FAST را ده‌ها برابر سریع‌تر از هریس می‌کند و به همین دلیل هستهٔ آشکارسازِ ORB است. بهایش این است که FAST به‌تنهایی مقیاس و جهت را نمی‌دهد؛ ORB این‌ها را با هرمِ تصویر و محاسبهٔ جهتِ گشتاور اضافه می‌کند.

۲ از نقطهٔ کلیدی تا توصیف‌گر

پیداکردنِ محلِ ویژگی (نقطهٔ کلیدی یا keypoint) تنها گامِ اول است. گامِ دوم این است که ظاهرِ آن نقطه را چطور توصیف کنیم تا بتوانیم آن را در تصویرِ دیگر بازبشناسیم. این کار با **توصیف‌گر** (descriptor) انجام می‌شود: برداری که وصلهٔ اطرافِ نقطهٔ کلیدی

را خلاصه می‌کند.

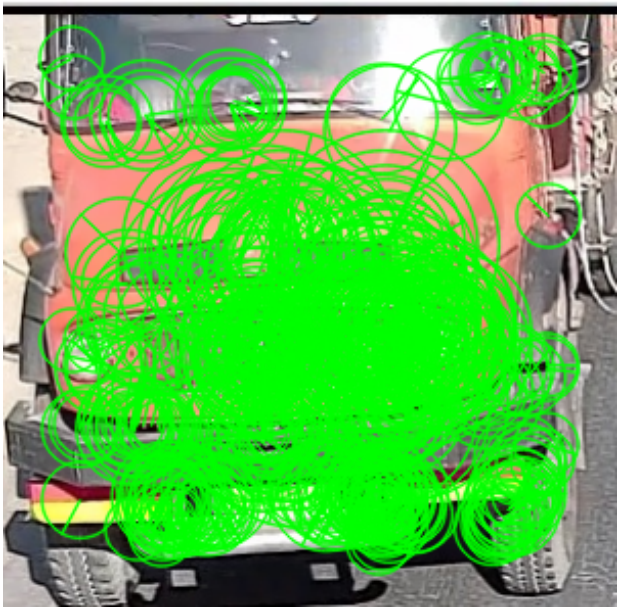
یادداشت

یک توصیف‌گرِ خوب باید **ناوردا** (invariant) باشد: مقدارش با چرخش، تغییرِ اندازه و تغییرِ روشنایی ثابت بماند. اگر توصیف‌گرِ یک نقطه در عکسِ صاف و عکسِ چرخیده یکسان باشد، می‌توانیم آن دو را مطابقت دهیم.

معروف‌ترین توصیف‌گرِ کلاسیک، **SIFT** است: یک بردارِ ۱۲۸بُعدی از هیستوگرام‌های جهتِ گرادیان، که در برابرِ چرخش، مقیاس و روشنایی بسیار مقاوم است. توصیف‌گرِ عملی‌ترِ ما **ORB** است.

۱.۲ ORB روی خودروی واقعی

ORB ترکیبی است از نقاطِ کلیدیِ FAST به‌علاوهٔ توصیف‌گرِ باینریِ BRIEF که آگاه‌به‌چرخش شده است. رایگان و سریع است. در شکلِ زیر، ۳۸۰ نقطهٔ کلیدیِ ORB روی همان خودرو را می‌بینید؛ هر دایره یک نقطهٔ کلیدی با اندازه (مقیاس) و جهت است:



۳۸۰ نقطهٔ کلیدیِ ORB؛ هر دایره یک ویژگیِ متمایز با توصیف‌گرِ باینری است

۲.۲ هریس در برابرِ شی-توماسی

شاید بپرسید چرا چند آشکارسازِ گوشه داریم. تفاوت در «امتیازی» است که هر کدام برای گوشه‌بودن حساب می‌کند:

- **هریس** از فرمولِ $R = \det(M) - k\,\mathrm{tr}(M)^2$ استفاده می‌کند که به پارامترِ حساسِ k نیاز دارد.
- **شی-توماسی** (تابعِ `goodFeaturesToTrack`) به‌جای آن مستقیماً کوچک‌ترین مقدارِ ویژه $\min(\lambda_1, \lambda_2)$ را امتیاز می‌دهد. منطقش این است: یک گوشهٔ خوب باید در هر دو جهت تغییرِ قوی داشته باشد، پس ضعیف‌ترین جهت را می‌سنجیم.

نکتهٔ کلیدی

شی–توماسی در عمل برای **ردیابی** (دنبال‌کردنِ نقاط در ویدیو) پایدارتر است و به همین دلیل در الگوریتم‌های ردیابیِ نوریِ (optical flow) کلاسیک محبوب است. برای ما هر دو نشان می‌دهند که «آشکارسازی» یک مسئلهٔ طراحیِ امتیاز است، نه یک جوابِ یکتا.

۳.۲ مقایسهٔ توصیف‌گرها

توصیف‌گر	نوع	ویژگی
SIFT	اعشاری ۱۲۸بُعدی	بسیار مقاوم؛ استانداردِ طلایی
SURF	اعشاری	شبیهِ SIFT اما سریع‌تر (دارای ثبتِ اختراع)
ORB	باینری ۲۵۶بیتی	رایگان، سریع، عالی برای بی‌درنگ
BRIEF	باینری	سریع اما به‌تنهایی ناوردابه‌چرخش نیست

نکتهٔ کلیدی

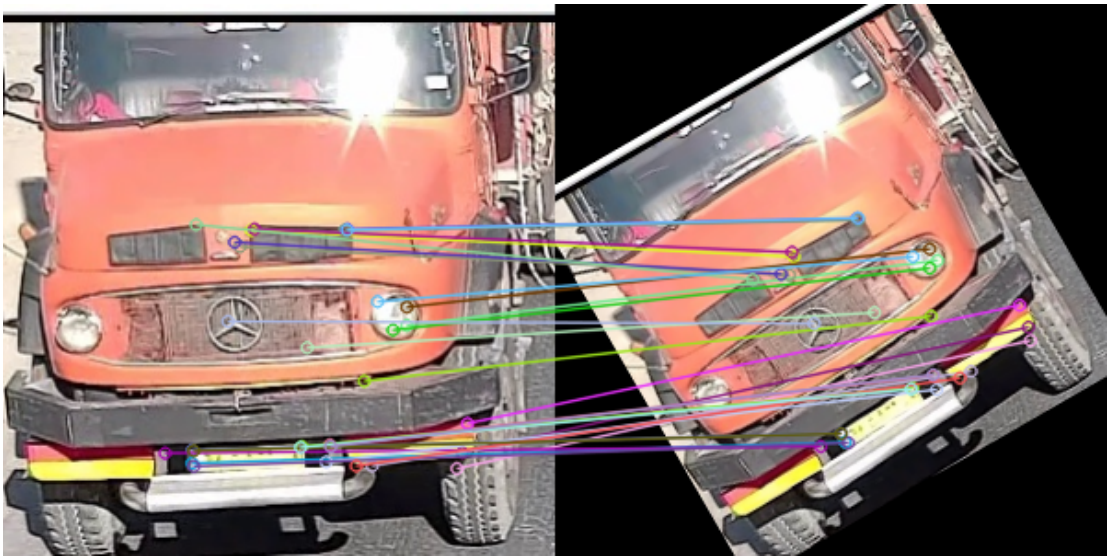
پیش‌فرضِ عملیِ ما ORB است: رایگان، سریع و به‌قدرِ کافی خوب. وقتی به بیشترین مقاومت نیاز داشتید (مثلاً تغییرِ ظاهرِ شدید)، سراغِ SIFT بروید.

۳ تطبیقِ توصیف‌گرها میانِ تصویرها

حالا که برای هر نقطهٔ کلیدی یک توصیف‌گر داریم، تطبیق ساده است: برای هر توصیف‌گر در تصویرِ A، نزدیک‌ترین توصیف‌گر در تصویرِ B را (بر اساسِ فاصله) پیدا می‌کنیم.

- برای توصیف‌گرهای اعشاری (SIFT): فاصلهٔ اقلیدسی.
- برای توصیف‌گرهای باینری (ORB): فاصلهٔ **همینگ** — تعدادِ بیت‌های متفاوت، که با یک XOR و شمارشِ بیت بسیار سریع محاسبه می‌شود.
- روشِ **جست‌وجوی فراگیر** (Brute-force) همهٔ جفت‌ها را مقایسه می‌کند؛ **FLANN** برای مجموعه‌های بزرگ این کار را تقریبی و سریع انجام می‌دهد.

در شکلِ زیر، همان خودرو را چرخانده و کوچک کرده‌ایم (چرخشِ ۳۰ درجه، مقیاسِ ۰٫۸) و ویژگی‌های ORB را بینِ نسخهٔ اصلی و نسخهٔ تغییریافته مطابقت داده‌ایم. خط‌ها نقاطِ فیزیکیِ یکسان را به هم وصل می‌کنند:



تطبیقِ ORB بینِ خودرو و نسخهٔ چرخیده/کوچک‌شدهٔ آن — ناوردایی در عمل، روی دادهٔ واقعی

این تصویر قلبِ درس است: با وجودِ چرخش و تغییرِ اندازه، ویژگی‌ها هنوز درست مطابقت می‌خورند. این همان چیزی است که به آن «ناوردایی» می‌گوییم.

۱.۳ یک مثالِ عددیِ فاصلهٔ همینگ

فرض کنید دو توصیف‌گرِ باینریِ ۸بیتی داریم (توصیف‌گرِ واقعیِ ORB ۲۵۶بیتی است، اما اصل یکسان است):

ریاضیات

$$d_1 = 10110010$$

$$d_2 = 10010110$$

$$d_1 \oplus d_2 = 00100100$$

تعدادِ بیت‌های ۱ در حاصلِ XOR برابرِ 2 است، پس فاصلهٔ همینگ $= 2$. یعنی این دو توصیف‌گر تنها در ۲ بیت از ۸ بیت تفاوت دارند — بسیار شبیه‌اند و احتمالاً یک تطبیقِ درست‌اند.

چون این محاسبه فقط یک XOR و یک شمارشِ بیت (popcount) است، پردازنده‌های امروزی آن را در یک دستور انجام می‌دهند؛ به همین دلیل تطبیقِ توصیف‌گرهای باینری بسیار سریع است.

۲.۳ کنارگذاشتنِ تطبیق‌های بد

بسیاری از تطبیق‌های نزدیک‌ترین‌همسایه غلط‌اند. باید فیلترشان کنیم:

- **آزمونِ نسبت (Ratio Test) — لُوو:** یک تطبیق را فقط وقتی نگه می‌داریم که بهترین همسایه به‌طورِ محسوسی از دومین‌همسایه نزدیک‌تر باشد (مثلاً فاصلهٔ اول $\times ۰٫۷۵ <$ فاصلهٔ دوم). اگر دو همسایه تقریباً هم‌فاصله باشند، تطبیق مبهم است و دور انداخته می‌شود.
- **بررسیِ متقابل (Cross-check):** فقط تطبیق‌هایی را نگه می‌داریم که در هر دو جهت نزدیک‌ترین همسایه باشند.
- **RANSAC (هفتهٔ ۱۱):** برون‌داده‌های هندسی را با برازشِ یک تبدیل کنار می‌گذارد.

هشدار / دام

تطبیق‌های خام پُرنویزاند. بدونِ آزمونِ نسبت یا RANSAC، حتی چند تطبیقِ غلط می‌تواند کلِ فرآیندِ انطباق را خراب کند. همیشه فیلتر کنید.

۳.۳ چرا آستانهٔ ۰٫۷۵ در آزمونِ نسبت؟

دیوید لُوو (مبتکرِ SIFT) با آزمایش روی هزاران تطبیق نشان داد که اگر نسبتِ فاصلهٔ نزدیک‌ترین‌همسایه به دومین‌نزدیک‌ترین را رسم کنیم، تطبیق‌های درست تقریباً همیشه نسبتِ کوچکی دارند (اولی خیلی نزدیک‌تر است)، در حالی که تطبیق‌های غلط نسبتی نزدیک به ۱ دارند (هر دو همسایه تصادفی و تقریباً هم‌فاصله‌اند). آستانهٔ حدودِ ۰٫۹۰ تطبیق‌های غلط را حذف می‌کند و تنها حدودِ ۵٪ تطبیق‌های درست را از دست می‌دهد — معامله‌ٔ بسیار خوبی.

یادداشت

نکتهٔ عملی: آزمونِ نسبت به `knnMatch` با $k = 2$ نیاز دارد (دو همسایهٔ نزدیک برای هر نقطه)، نه `match` با بررسیِ متقابل. این دو راهبردِ فیلترکردن‌اند و معمولاً یکی را انتخاب می‌کنید.

۴ ناوردایی به مقیاس: هرمِ تصویر

چطور یک ویژگی را در هر اندازه‌ای پیدا کنیم؟ با ساختنِ **هرمِ تصویر** (image pyramid): مجموعه‌ای از نسخه‌های به‌تدریج کوچک‌ترِ همان تصویر. ویژگی‌ها را در هر مقیاس آشکار می‌کنیم؛ مقیاسی که در آن ویژگی قوی‌ترین پاسخ را می‌دهد، «مقیاسِ مشخصهٔ» آن ویژگی است. به همین ترتیب SIFT و ORB می‌توانند یک الگوی کوچک را با یک عکسِ بزرگ مطابقت دهند.

ریاضیات

برای برآوردِ یک **هوموگرافی** (تبدیلِ پرسپکتیو، هفتهٔ ۱۱) به دستِ‌کم تطبیقِ خوب نیاز است. در عمل ده‌ها تطبیق جمع می‌کنیم و می‌گذاریم RANSAC یک زیرمجموعهٔ سازگارِ ۴تایی (یا بیشتر) را انتخاب کند. هرچه تطبیق‌های خوبِ بیشتری داشته باشیم، تبدیلِ برآوردشده پایدارتر و دقیق‌تر است.

۵ راهبردهای تطبیق در یک نگاه

سه راهبردِ رایج برای تطبیق و فیلترکردن وجود دارد؛ انتخابِ درست به کاربردِ شما بستگی دارد:

راهبرد	چگونه	کِی استفاده
بررسیِ متقابل	`BFMatcher(crossCheck=True)`	ساده، سریع، تطبیقِ یک‌به‌یک
آزمونِ نسبت	`knnMatch(k=2)` + آستانه	دقیق‌ترین فیلتر؛ توصیهٔ لُوو
FLANN	درختِ تقریبی	مجموعه‌های خیلی بزرگِ ویژگی

هشدار / دام

یک اشتباهِ رایج: استفادهٔ هم‌زمان از `crossCheck=True` و `knnMatch`. این دو با هم ناسازگارند؛ `knnMatch` برای هر نقطه چند همسایه می‌خواهد، اما `crossCheck` فقط یک تطبیقِ متقابل برمی‌گرداند. یکی را انتخاب کنید.

۱.۵ اشتباه‌های رایجِ دانشجویان

- **فراموشیِ تبدیل به خاکستری:** آشکارسازها روی تصویرِ تک‌کاناله کار می‌کنند؛ دادنِ تصویرِ رنگی گاهی خطا یا نتیجهٔ ضعیف می‌دهد.
- **نگه‌داشتنِ همهٔ تطبیق‌ها:** بدونِ فیلتر، تطبیق‌های غلط کلِ انطباق را خراب می‌کنند. همیشه فیلتر کنید.
- **انتظارِ ویژگی روی سطحِ صاف:** روی یک برگهٔ سفیدِ بی‌بافت، ORB نقاطِ کمی پیدا می‌کند؛ باید به لبه‌ها و بافتِ کادر تکیه کرد.
- **مقایسهٔ توصیف‌گرهای ناهم‌جنس:** فاصلهٔ همینگ فقط برای توصیف‌گرهای باینری (ORB/BRIEF) است؛ برای SIFT از `NORM_L2` استفاده کنید.

۶ کاربردها

- **دوختنِ تصویر (پانوراما):** ویژگی‌ها را مطابقت بده، تبدیل را پیدا کن، تصویرها را ادغام کن.
- **شناساییِ شیء:** ویژگی‌های یک الگو را با صحنه مطابقت بده.
- **ردیابی، SLAM و واقعیتِ افزوده:** ویژگی‌ها را در طولِ ویدیو دنبال کن تا حرکت را برآورد کنی.
- **DocScan:** صفحه/الگو را مطابقت بده تا آن را منطبق و صاف کنی (هفتهٔ ۱۱).

۱.۶ ویژگی‌های کلاسیک در برابرِ عمیق

ویژگی‌های کلاسیک (SIFT/ORB) دست‌طراحی، قابلِ‌تفسیر و بدونِ نیاز به آموزش‌اند و هنوز برای مسائلِ هندسی عالی عمل می‌کنند. توصیف‌گرهای عمیق (مثلِ SuperPoint) در تغییرِ ظاهرِ شدید بهتر عمل می‌کنند اما به آموزش نیاز دارند. برای انطباق و هندسه، ویژگی‌های کلاسیک همچنان انتخابی قوی و عملی هستند.

نکتهٔ کلیدی

جمع‌بندیِ مفهومی: ویژگی راهی است که کامپیوتر بگوید «این نقطه در اینجا، همان نقطه در آنجاست». همین توانایی، بنیادِ همهٔ بینایی هندسی است.

۲.۶ سناریوی DocScan: منطبق‌کردنِ عکسِ کنتور روی الگو

بیایید کاربردِ مستقیمِ این جلسه در پروژه را قدم‌به‌قدم ببینیم. فرض کنید یک «الگو» (تصویرِ تمیزِ روبه‌رو از صفحهٔ کنتور) داریم و کاربر با موبایل عکسی کج و از زاویه گرفته است:

۱. روی هر دو تصویر ویژگی‌های ORB را آشکار و توصیف می‌کنیم.

۲. توصیف‌گرها را با فاصلهٔ همینگ مطابقت می‌دهیم و با آزمونِ نسبت فیلتر می‌کنیم؛ چند ده تطبیقِ خوب می‌ماند.

۳. این تطبیق‌ها می‌گویند «گوشهٔ بالاچپِ کادرِ کنتور در الگو، در عکس کجاست»، و همین‌طور برای بقیهٔ نقاط.

۴. هفتهٔ آینده با `findHomography` یک تبدیلِ پرسپکتیو از این تطبیق‌ها برآورد می‌کنیم و با `warpPerspective` عکسِ کج را به نمای روبه‌رو و صاف تبدیل می‌کنیم — آمادهٔ خواندنِ ارقام.

نکتهٔ کلیدی

پس ویژگی‌ها «چسبِ هندسی» :اندDocScan بدونِ آن‌ها نمی‌دانیم کدام نقطهٔ عکس با کدام نقطهٔ الگو متناظر است، و بدونِ این تناظر نمی‌توان تصویر را صاف کرد.

۷ کد: از آشکارسازی تا تطبیق

```
import cv2
img1 = cv2.imread('meter.jpg', 0)
img2 = cv2.imread('meter_tilted.jpg', 0)

orb = cv2.ORB_create(500)
```

```python
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

# match with cross-check
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)

# draw the best 30
out = cv2.drawMatches(img1, kp1, img2, kp2, matches[:30], None)
cv2.imwrite('matches.png', out)
# next week: cv2.findHomography(pts1, pts2, cv2.RANSAC) to align
```

الگوی همیشگی: آشکارسازی ← توصیف ← تطبیق ← (هفتهٔ ۱۱) برآوردِ تبدیل.

۸ چطور کیفیتِ ویژگی‌ها را بسنجیم؟

وقتی روی داده خودتان کار می‌کنید، از کجا بدانید آشکارسازتان خوب عمل می‌کند؟ دو معیارِ عملی:

- **تکرارپذیری (Repeatability):** تصویر را کمی بچرخانید یا مقیاسش را عوض کنید؛ چه کسری از نقاطِ کلیدی دوباره (تقریباً در همان محلِ فیزیکی) پیدا می‌شوند؟ هرچه بیشتر، بهتر.
- **نرخِ تطبیقِ درست (Precision):** از تطبیق‌هایی که الگوریتم برمی‌گرداند، چه کسری واقعاً درست‌اند؟ این را می‌توان با دانستنِ تبدیلِ واقعی (مثلاً وقتی خودتان تصویر را چرخانده‌اید) سنجید.

ریاضیات

در مثالِ خودروی این جلسه، ما تبدیل را می‌دانستیم (چرخشِ ۳۰ درجه، مقیاسِ ۰٫۸)، پس می‌توانستیم بررسی کنیم که آیا هر تطبیق با این تبدیل سازگار است یا نه. همین ایده، هستهٔ RANSAC در هفتهٔ آینده است: تبدیل را نمی‌دانیم، پس آن را از خودِ تطبیق‌ها حدس می‌زنیم و تطبیق‌های ناسازگار را دور می‌ریزیم.

یادداشت

درسِ مهندسی: هر جزءِ سامانه را با یک معیارِ کمی بسنجید. «به‌نظر خوب می‌آید» کافی نیست؛ «۹۲٪ تطبیق‌ها درست‌اند» قابلِ اتکا و قابلِ بهبود است.

۹ کدِ کامل‌تر: آزمونِ نسبت

```python
# ratio test (Lowe) - the recommended filter
bf = cv2.BFMatcher(cv2.NORM_HAMMING)    # no crossCheck with knnMatch
knn = bf.knnMatch(des1, des2, k=2)      # 2 nearest neighbours each
good = []
for m, n in knn:
    if m.distance < 75.0 * n.distance: # best clearly closer than 2nd
        good.append(m)
print(f"kept {len(good)} of {len(knn)} matches")
```

۱۰ تمرینِ کلاسی و پاسخ

تمرین

۱) چرا یک گوشه ویژگیِ بهتری از یک نقطه در وسطِ دیوارِ صاف است؟

۲) یک الگوی پلاک را با عکسی که از زاویه و نصفِ اندازه گرفته شده مطابقت می‌دهید؛ کدام خاصیتِ توصیف‌گر این کار را ممکن می‌کند؟

۳) پس از تطبیق، چند خط به‌وضوح غلط‌اند (درهم‌رفته). چطور حذفشان می‌کنید؟

پاسخ‌ها: (۱) گوشه در همهٔ جهت‌ها تغییر می‌کند، پس به‌طورِ یکتا مکان‌یابی می‌شود؛ نقطهٔ صاف مبهم است. (۲) ناوردایی به مقیاس و چرخش — توصیف‌گر با وجودِ تغییرِ اندازه و زاویه یکسان می‌ماند. (۳) آزمونِ نسبت و بررسیِ متقابل بیشترشان را حذف می‌کنند؛ RANSAC (هفتهٔ ۱۱) برون‌داده‌های هندسی را با برازشِ یک تبدیل کنار می‌گذارد.

۱۱ زمینهٔ تاریخی و پیوند با ادامهٔ درس

آشکارسازِ گوشهٔ هریس در سالِ ۱۹۸۸ معرفی شد و SIFT در سالِ ۱۹۹۹ توسطِ دیوید لُوو — این‌ها انقلابی در بینایی ماشین بودند، چون برای اولین بار امکانِ تطبیقِ قابلِ‌اعتماد بینِ تصویرهای بسیار متفاوت را فراهم کردند. تا پیش از دورانِ یادگیریِ عمیق (حدودِ ۲۰۱۲)، تقریباً هر سامانهٔ شناساییِ شیء بر پایهٔ همین ویژگی‌های دست‌طراحی بنا شده بود. امروز هم برای مسائلِ هندسی (انطباق، دوخت، بازسازیِ سه‌بُعدی) این ویژگی‌ها همچنان انتخابِ اول‌اند، چون سریع، قابلِ‌تفسیر و بی‌نیاز از آموزش‌اند.

یادداشت

نقشهٔ راهِ پروژه: هفتهٔ ۹ کانتورِ بیرونیِ سند را به ۴ گوشه تقریب زدیم؛ این هفته یاد گرفتیم نقاطِ متناظر را بینِ دو تصویر پیدا کنیم؛ هفتهٔ ۱۱ از این تناظرها یک تبدیلِ پرسپکتیو می‌سازیم و تصویر را صاف می‌کنیم؛ و در هفته‌های ۱۲ تا ۱۴ ارقامِ صاف‌شده را با یادگیریِ ماشین و شبکه‌های عصبی می‌خوانیم. هر هفته یک حلقه از همین زنجیره است.

هم‌چنین توجه کنید که ویژگی‌ها پلی به سمتِ یادگیریِ ماشین هم هستند: در هفتهٔ ۱۲ خواهیم دید که می‌توان از توصیف‌گرها به‌عنوانِ «ویژگیِ ورودی» برای یک طبقه‌بند استفاده کرد — درست همان‌طور که یک انسان اجسام را با نشانه‌های متمایزشان تشخیص می‌دهد.

۱۲ جمع‌بندی

در این جلسه یاد گرفتیم ویژگیِ خوب چیست (متمایز، تکرارپذیر، محلی) و چرا گوشه‌ها انتخابِ کلاسیک‌اند؛ گوشه‌ها را با هریس/FAST آشکار کردیم و توصیف‌گر را با ORB/SIFT ساختیم؛ توصیف‌گرها را مطابقت دادیم و تطبیق‌های بد را با آزمونِ نسبت و بررسیِ متقابل حذف کردیم؛ و دیدیم ناوردایی به مقیاس و چرخش چطور روی دادهٔ واقعی کار می‌کند. هفتهٔ آینده از همین تطبیق‌ها برای برآوردِ هوموگرافی و صاف‌کردنِ عکسِ سند/کنتور در DocScan استفاده می‌کنیم.